# Faculty Induction System

## Project Analysis Report

| | | | |
|---|---|---|---|
| **Project Type** | Full-Stack Web Application | **Architecture** | Frontend + Backend API |
| **Report Date** | April 22, 2026 | **Classification** | Internal / Confidential |

## ■ 1. Executive Summary

This report presents a comprehensive technical analysis of the **Faculty Induction System**, a recruitment platform designed to streamline the end-to-end faculty hiring process. Candidates submit structured applications encompassing academic qualifications, professional work experience, and research publications. Administrators then review, numerically score, and track each candidate through a clearly defined hiring pipeline.

The system adopts a modern, decoupled full-stack architecture, separating concerns cleanly between a React-based single-page application on the frontend and a RESTful .NET 8.0 API on the backend, persisted via SQLite.

| | | | | |
|---|---|---|---|---|
| **React + Vite** | **C# .NET 8.0** | **SQLite** | **JWT** | **Tailwind CSS** |
| Frontend | Backend | Database | Auth | Styling |

## ■ 2. Core Technologies & Purpose

| Technology | Purpose in Project |
|---|---|
| **C# .NET 8.0** | Backend runtime; powers API endpoints, business logic, and scoring algorithms |
| **ASP.NET Core** | RESTful API framework with 6 controllers: Auth, Users, Forms, Admin |
| **Entity Framework Core** | ORM eliminating manual SQL; maps models to tables; handles migrations automatically |
| **SQLite** | Zero-configuration local database (FacultyInduction.db) |
| **JWT** | Stateless authentication; stores user ID, email, and role in signed token |
| **BCrypt** | One-way password hashing with automatic salt generation |
| **React 18** | Component-based UI; 12 pages; state management with hooks |
| **React Router** | Client-side routing; protected routes for admin/applicant separation |
| **Vite** | Fast build tool; dev server with API proxy to backend |
| **Tailwind CSS** | Utility-first styling; responsive design; custom color palette |
| **Axios** | HTTP client; interceptor auto-adds JWT token to all outgoing requests |
| **React Hook Form + Zod** | Form validation with minimal re-renders; schema-based validation rules |
| **SMTP Email** | Password reset functionality with 1-hour expiry token delivery |

## ■ 3. Key Functionalities

### 3.1 Authentication

| Feature | Endpoint | Key Detail |
|---|---|---|
| Registration | POST /api/auth/register | BCrypt hashing; default 'Applicant' role assigned |
| Login | POST /api/auth/login | JWT token returned and stored in localStorage |
| Forgot Password | POST /api/auth/forgot-password | 1-hour reset token delivered via SMTP email |

### 3.2 Role-Based Feature Sets

**Applicant Features**

- Dashboard with profile summary and status
- Academic Qualifications form (Matric through PhD)
- Work Experience form with date tracking
- Research Publications form
- Score Card showing calculated total
- Profile management with image upload

**Admin Features**

- Dashboard with statistics: submitted, pending, shortlisted, hired, rejected
- Candidate list sorted by score
- Detailed application review with score breakdown
- Status management: Pending → Shortlisted → Hired / Rejected

### 3.3 Scoring Engine

| Component | Max Score | Notes |
|---|---|---|
| Academic Qualifications (Matric to PhD + GPA Bonus) | **50** | Scaled by degree level and GPA performance |
| Work Experience (varies by position & hiring type) | **5 – 20** | Ranges based on seniority and contract type |
| Research Publications | **Up to 30** | Impact factor and journal quality considered |

## ■■ 4. Database Schema

The system uses **5 core relational tables** in SQLite, with one-to-many relationships from the Users table to qualifications, experiences, and publications, and a one-to-one relationship to ApplicationRecords.

| Table | Purpose | Key Columns |
|---|---|---|
| **Users** | Authentication, profile, and role management | id, email, passwordHash, role, profileImage |
| **ApplicationRecords** | Hiring type, position, total score, pipeline status | id, userId, hiringType, position, totalScore, status |
| **AcademicQualifications** | Degree, institute, GPA, passing year | id, userId, degree, institute, gpa, passingYear |
| **WorkExperiences** | Organization, position, dates, current-job flag | id, userId, organization, position, startDate, endDate, isCurrent |
| **ResearchPublications** | Title, journal, publication date, impact factor | id, userId, title, journal, publicationDate, impactFactor |

**Relationships:** One User → Many AcademicQualifications / WorkExperiences / ResearchPublications | One User → One ApplicationRecord

## ■ 5. Security Measures

| Measure | Implementation |
|---|---|
| **Password Storage** | BCrypt hashing with per-user salt; plaintext never persisted |
| **Authentication** | JWT signed with 32-character secret key; short-lived tokens |
| **Authorization** | [Authorize] and [Authorize(Roles='Admin')] ASP.NET Core attributes |
| **Password Reset** | Single-use token with 1-hour expiry; invalidated on use |
| **Email Uniqueness** | Unique constraint enforced at the database level in SQLite |

## ■ 6. API Endpoints Summary

| Method | Endpoint | Role |
|---|---|---|
| POST | /api/auth/register | Public |
| POST | /api/auth/login | Public |
| POST | /api/auth/forgot-password | Public |
| POST | /api/forms/academic-qualifications | Applicant |
| GET | /api/forms/academic-qualifications | Applicant |
| POST | /api/forms/work-experiences | Applicant |
| GET | /api/forms/work-experiences | Applicant |

| | | |
|---|---|---|
| POST | /api/forms/research-publications | Applicant |
| GET | /api/forms/research-publications | Applicant |
| PUT | /api/users/profile | Applicant |
| GET | /api/admin/applications | Admin |
| GET | /api/admin/applications/{id} | Admin |
| PUT | /api/admin/applications/{id}/status | Admin |
| GET | /api/admin/stats | Admin |

*This document was auto-generated for internal review purposes. All architectural decisions, endpoint definitions, and security implementations are subject to revision as the project evolves.*